FOUNDATIONAL MODELS & GENERATIVE AI

EXAM-READY STUDY GUIDE

Built from Lecture Transcripts • Lectures 1 – 15

Prepared for the M.Tech AI study group • IIT Jodhpur

CONTENTS

L1	**The Lay of the Land**	*Why generative AI, deep learning recap, course shape*
L2	**Generative Models & GANs**	*Generator vs discriminator, training dynamics*
L3	**Autoencoders & VAEs**	*Latent space, reparameterization trick, KL divergence*
L4	**The Transformer**	*Q, K, V, attention as a lookup, why it replaced RNNs*
L5	**Self-Attention vs Cross-Attention**	*Where Q, K, V come from — the only thing that changes*
L6	**Diffusion & Stable Diffusion**	*Forward noising, reverse denoising, DDPM algorithm*
L7	**Embeddings, LLMs, VLMs**	*Vector spaces, model families, the modality bridge*
L8	**BERT**	*Bidirectional encoder, MLM, NSP, fine-tuning*
L9	**GPT & State Space Models**	*Decoder-only generation, SSM/Mamba alternatives*
L10	**Contrastive Learning (SimCLR)**	*Pull similars together, push others apart*
L11	**Pre-training & Fine-tuning**	*Transfer learning, SFT, PEFT motivation*
L12	**LoRA**	*Low-rank adapters — the math, the savings*
L13	**QLoRA & RAG**	*4-bit quantization, retrieval-augmented generation*
L14	**Flash Attention**	*IO-awareness, tiling, online softmax*
L15	**Prompt Engineering & Risks**	*Temperature, top-p, hallucinations, bias*
★	**Master Cheat Sheet**	*Everything in two pages*
★	**Final Flashcard Sprint**	*60 cards across all topics*

LECTURE

01

The Lay of the Land

Why generative AI matters, deep learning recap, course shape

CORE IDEA

Deep learning learns representations of data with multiple levels of abstraction. Generative AI is the next step: instead of predicting a label from data, we learn the data distribution itself and sample new data from it. The course is built around three model families that did this best — GANs, VAEs/diffusion, and transformers — and the techniques (LoRA, QLoRA, RAG, Flash Attention) that made them practical.

DEEP DIVE

The professor framed the course around what changed in deep learning over the past decade. Three forces converged: better hardware (GPUs, then NVIDIA's purpose-built training chips), much larger datasets (ImageNet has roughly one million training images across thousands of categories), and architectural breakthroughs (better activations, normalization, residual connections, then attention).

The ImageNet challenge is the canonical illustration. Early models had error rates around 28%; modern models reach about 3.5% — below human performance for the same task. The trajectory of that error curve is the trajectory of deep learning itself.

Generative AI then takes this further. A discriminative model answers "is this a cat?" A generative model answers "give me a new image that looks like a cat I have never seen." That requires modeling the full data distribution, which is much harder, and is the whole subject of the course.

INTUITION

Classification asks the model to draw a line. Generation asks the model to paint the whole canvas. The math, training dynamics, and failure modes are completely different — that is why generative AI deserves its own course rather than being a chapter in a deep learning class.

COURSE MAP

The professor laid out the syllabus in roughly this order: GANs and autoencoders (week 1–2), then transformers and attention (week 2–3), then diffusion models (week 3), then LLMs and VLMs with BERT and GPT (week 4), then state space models as a transformer alternative, then contrastive learning, then the production stack — fine-tuning, LoRA, QLoRA, RAG, Flash Attention — and finally prompt engineering with hallucinations and bias.

WHAT THE PROF EMPHASIZED

"I will not go into the details of deep learning as such — if you don't know how to train a model, this course will be very tough." The prof assumes you can already write a training loop, define a loss, and use backprop. This guide assumes the same.

Course assessment: end-term 40, mid-term 20, three assignments + three quizzes, plus a 10-mark viva (an individual video call with the TAs).

QUICK RECALL

Q	A
What is the central distinction between discriminative and generative models?	Discriminative models learn p(y | x) — predict a label from input. Generative models learn p(x) (or p(x, y)) — model the data distribution itself, so you can sample new x.
Why did deep learning take off around 2012?	Three things converged: GPUs made training fast, ImageNet-scale labeled data became available, and architectures (deeper nets, ReLU, dropout, later residuals) made optimization stable.
What was the ImageNet error rate trajectory?	Roughly 28% in early years to about 3.5% today — passing human-level performance on the benchmark.
Name the three main families this course covers.	Likelihood-based (VAEs, diffusion), implicit (GANs), and autoregressive transformers (GPT-style).

LECTURE

02

Generative Models & GANs

Generator vs discriminator, the minimax game, training instability

CORE IDEA

Generative modeling means: given many examples drawn from some distribution (say, photographs of horses), learn to produce new samples that look like they came from the same distribution. A GAN — Generative Adversarial Network — does this by pitting two networks against each other: a generator that produces fake samples from random noise, and a discriminator that tries to tell real from fake. As the discriminator gets better, the generator is forced to get better, and at equilibrium the generator has learned the true data distribution.

DEEP DIVE

The setup is a two-player game. The generator G takes a noise vector z (sampled from a simple prior like a standard normal) and outputs a fake sample G(z). The discriminator D takes any sample — either a real one from the dataset or a fake one from G — and outputs a probability that the sample is real.

During training they update alternately. The discriminator is trained to maximize the probability it assigns to real samples and minimize the probability it assigns to fakes — a standard binary classification loss. The generator is trained to fool the discriminator: it wants D(G(z)) to be close to 1, i.e. it wants the discriminator to think its fakes are real.

FORMULA

```
min_G  max_D   E_x[ log D(x) ]  +  E_z[ log(1 - D(G(z))) ]
```

Read the objective from the inside out. The discriminator is the inner max — for any fixed G, D wants to push log D(x) up on real data and push log(1 − D(G(z))) up on fake data (which means pushing D(G(z)) down). The generator is the outer min — given the best D, G wants to make log(1 − D(G(z))) as small as possible, i.e. make D(G(z)) as large as possible. At the global optimum, G has matched the data distribution and D outputs 0.5 everywhere because it cannot tell the two apart.

INTUITION

Think of a forger and a detective. The forger paints fake portraits; the detective examines each painting and decides whether it is genuine. Each time the detective catches a forgery, the forger learns what gave them away and improves. Each time a forgery slips through, the detective sharpens their eye. Eventually the forger is producing portraits indistinguishable from the masters — and the detective is reduced to a coin flip.

WHY GANS ARE HARD TO TRAIN

- Mode collapse: the generator finds one or two outputs that fool the discriminator and produces only those, ignoring the rest of the distribution. You get a model that generates the same horse a thousand different ways.
- Vanishing generator gradient: if the discriminator gets too good too fast, log(1 − D(G(z))) saturates and the generator gets no useful signal. The standard fix is to have G maximize log D(G(z)) instead — same fixed point, much better gradients early in training.
- Non-convergence: unlike normal supervised learning, there is no single loss going down. The two losses oscillate as both networks chase each other. You cannot tell from the loss curve alone whether training is going well — you have to look at samples.
- Hyperparameter sensitivity: learning rates, batch sizes, and architectural choices (DCGAN's specific recipe of strided convolutions, batch norm, leaky ReLU) matter much more than in standard CNNs.

WATCH OUT

A common student mistake: looking at the GAN loss curves and concluding training is broken because they oscillate. They are supposed to oscillate. Judge a GAN by its samples, not its loss.

Another: assuming the generator's noise prior z encodes meaning. It does not, in vanilla GANs. The mapping from z to image is whatever the generator learned — usually entangled and uninterpretable. Disentangled representations need extra machinery (InfoGAN, StyleGAN, etc.).

EXAM TIP

If asked to write the GAN objective, write the full minimax expression with both expectations. Half marks usually for getting the structure (one log on real, one log of one-minus on fake) even if signs are mixed up.

If asked why GANs collapse, the answer is mode collapse — and the cause is that the generator's only signal is the discriminator's gradient, which does not penalize lack of diversity directly.

QUICK RECALL

Q In a GAN, what does the generator take as input and produce as output?	**A** Input: a random noise vector z (typically from a standard normal). Output: a synthetic sample G(z) in the data space — e.g. an image.
Q What loss does the discriminator minimize?	**A** A standard binary cross-entropy: real samples should be classified as 1, generated samples as 0.
Q State the GAN minimax objective.	**A** min_G max_D E_x[log D(x)] + E_z[log(1 − D(G(z)))]
Q	**A**

What is mode collapse?	When the generator produces only a small subset of the data distribution — fooling the discriminator on a few easy samples while ignoring the rest of the modes.
Q At the global optimum of a GAN, what does D output on every input?	**A** 0.5 — because at equilibrium the generator's distribution matches the data distribution and the discriminator cannot distinguish them.
Q Why is the alternative generator loss log D(G(z)) preferred over log(1 − D(G(z)))?	**A** Same fixed point, but it has much stronger gradients when D is confident (early in training), avoiding the vanishing gradient that kills the original formulation.

LECTURE

03

Autoencoders & Variational Autoencoders

Compressing the world into a latent space, then sampling from it

CORE IDEA

An autoencoder squeezes an input through a narrow bottleneck and tries to reconstruct it on the other side. The bottleneck forces the network to learn a compressed representation — a latent code — that captures the essential structure of the data. A variational autoencoder (VAE) adds a twist: instead of mapping each input to a single point in latent space, it maps it to a distribution. This makes the latent space smooth and samplable, which turns the VAE from a compressor into a generative model.

PLAIN AUTOENCODER

Architecture: an encoder f maps input x to a latent code $z = f(x)$, where z lives in a much lower-dimensional space than x. A decoder g maps z back to a reconstruction $\hat{x} = g(z)$. Train end-to-end on reconstruction loss — typically mean squared error between x and $\hat{x}$.

The bottleneck is the whole point. If z had as many dimensions as x, the network could learn the identity function and the latent space would be useless. By forcing z to be small (say, 32 dimensions for an image with thousands of pixels), the encoder must throw away noise and keep only the structure that the decoder needs to reconstruct.

INTUITION

An autoencoder is like asking someone to describe a movie in three sentences and then asking someone else to recreate the movie from those three sentences. The first person learns to compress; the second learns to decompress. The three sentences are the latent code — they had better contain the important stuff.

Plain autoencoders are excellent for denoising, dimensionality reduction, and pretraining — but they are not generative. If you sample a random point in their latent space and decode it, you usually get garbage. The reason is that the encoder maps the training data to scattered, unconstrained points in latent space; most of the space is empty, and points outside the training cloud decode to nonsense.

VARIATIONAL AUTOENCODER

The VAE fixes this. Instead of mapping x to a single point z, the encoder outputs the parameters of a distribution over z — specifically, a mean $\mu(x)$ and a (log) variance $\sigma^2(x)$ of a Gaussian. The latent code is then sampled: $z \sim N(\mu(x), \sigma^2(x))$. The decoder reconstructs from this sampled z.

That sampling step is the heart of the difference. By forcing each input to occupy a small Gaussian-shaped region of latent space, and by adding a regularizer that pulls every input's distribution toward a shared standard normal prior, the VAE makes the latent space smooth: nearby points decode to similar outputs, and the whole space ends up roughly Gaussian, so you can sample from N(0, I) and get plausible new samples.

FORMULA

$$L = \underbrace{E_q[\log p(x \mid z)]}_{\text{reconstruction}} - \underbrace{KL(\, q(z|x) \;\|\; p(z)\,)}_{\text{regularizer}}$$

The loss has two terms. The first is the reconstruction term: how well does the decoder reproduce x from a z sampled from the encoder. The second is the KL divergence between the encoder's distribution q(z|x) and the prior p(z) = N(0, I); it penalizes the encoder for placing inputs far from the origin or with wildly different variances. Together they balance fidelity (reconstruct well) against organization (keep latent space tidy).

THE REPARAMETERIZATION TRICK

Sampling $z \sim N(\mu, \sigma^2)$ is non-differentiable — you cannot backpropagate through a random number. The trick is to rewrite the sample as a deterministic function of the parameters and an external source of noise:

FORMULA

$$z = \mu(x) + \sigma(x) \cdot \varepsilon \qquad \text{where} \quad \varepsilon \sim N(0, I)$$

Now the randomness lives in ε, which has no parameters. The path from μ and σ to z is a simple sum and product, both differentiable. Gradients flow through μ(x) and σ(x) back into the encoder. The professor stressed this point: the gradient flows freely because the randomness is no longer a function of the network parameters.

WHAT THE PROF EMPHASIZED

"We are not sampling a distribution. We are taking a noise and transforming it into a sample." — that one sentence is the reparameterization trick.

Encoder and decoder are trained together, end to end. There is no separate stage.

VAE VS GAN — THE CHEAT-SHEET VIEW

Aspect	VAE	GAN
Training objective	Maximize likelihood lower bound (ELBO)	Minimax adversarial game
Latent space	Smooth, organized, samplable	Implicit, often entangled

Sample quality	Slightly blurry (averaging effect)	Sharp, but unstable training
Mode coverage	Covers all modes (likelihood-based)	Prone to mode collapse
Inference (x → z)	Built in via the encoder	Not natively supported
Stability	Stable, well-behaved	Notoriously fragile

EXAM TIP

If asked to derive the VAE loss, start from "maximize log p(x)", introduce a variational distribution q(z|x), apply Jensen's inequality, and out pops the ELBO with reconstruction and KL terms.

If asked why VAEs produce blurry samples: the per-pixel reconstruction loss (MSE / Gaussian likelihood) effectively averages over many plausible reconstructions, which produces blurry means. GANs avoid this because their loss does not measure pixel-wise distance.

QUICK RECALL

Q What does the bottleneck in an autoencoder force the model to learn?	**A** A compressed representation that keeps only what is needed to reconstruct — essentially, the structure of the data manifold.
Q Why is a plain autoencoder not generative?	**A** Its latent space is unconstrained — sampling random latent points usually gives garbage because most of latent space lies outside the training cloud.
Q What does the VAE encoder output instead of a single point?	**A** The mean and (log) variance of a Gaussian distribution over latent codes.
Q Write the VAE loss in one line.	**A** ELBO = E_q(z|x)[log p(x|z)] − KL(q(z|x) ‖ p(z))
Q What is the reparameterization trick and why is it needed?	**A** Rewriting $z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim N(0, I)$ makes sampling differentiable — gradients can now flow through μ and σ back into the encoder, which they cannot do through a raw random sample.
Q Why are VAE samples typically blurrier than GAN samples?	**A** The per-pixel reconstruction loss averages over many plausible reconstructions, which gives blurred means. GANs use an adversarial loss that does not have this averaging effect.

LECTURE

04

The Transformer

Q, K, V, and why attention replaced recurrence

CORE IDEA

The transformer replaced recurrent networks for sequence modeling. Instead of processing tokens one at a time and carrying state forward, the transformer looks at every token in the sequence simultaneously and uses attention to decide which tokens are relevant to which. Three vectors per token — Query, Key, Value — power the entire mechanism. The result is a model that parallelizes beautifully, captures long-range dependencies without forgetting, and scales to sizes that RNNs never could.

WHY NOT RNNS?

RNNs process sequences left to right, updating a hidden state at each step. Two problems become fatal at scale. First, training is sequential — you cannot start step t until step t−1 is done — so RNNs do not benefit much from parallel hardware. Second, by the time information from token 1 reaches token 1000, it has been squashed through 999 nonlinearities and a fixed-size hidden state; long-range dependencies degrade. LSTMs and GRUs help but do not solve the problem.

The transformer's answer: forget recurrence entirely. Let every token directly attend to every other token in one step. The cost is quadratic in sequence length (which is exactly what Flash Attention will later address), but the wins are decisive — full parallelism in training and a direct path between any two positions.

Q, K, V — THE THREE VECTORS

For each input token, the model computes three vectors by linear projection from the token's embedding:

- **Query (Q):** what am I looking for? — the question this token is asking of the rest of the sequence.
- **Key (K):** what do I contain? — the label that other tokens' queries can match against.
- **Value (V):** what do I actually carry? — the content that gets passed on if this token is attended to.

Each token computes a Q, a K, and a V. To compute attention, we compare every token's Q against every token's K (a dot product), normalize the scores into a probability distribution with softmax, and use those probabilities to take a weighted sum of the V vectors. The output for each token is a blend of all the value vectors, weighted by how relevant each other token is to it.

FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Walk through the dimensions. If your sequence has n tokens and each Q, K, V is d-dimensional, then Q is n × d, K is n × d, V is n × d. The product $Q \cdot K^T$ is n × n — one similarity score for every (query, key) pair. Softmax along each row turns each row into a probability distribution over the n tokens. Multiplying that n × n attention matrix by the n × d V matrix gives an n × d output: each row is a weighted sum of value vectors.

INTUITION

Imagine a library lookup. Each book has a title written on its spine (the key). You walk in with a question (the query). You scan all the spines, and the books whose titles match your question best are the ones you pull off the shelf. What you actually read is what is inside the books (the value). Self-attention is exactly this lookup, but soft — instead of pulling one book, you take a weighted blend of all of them based on how well each spine matches your question.

WHY DIVIDE BY √D_K?

Without the scaling, when d_k is large, the dot products $Q \cdot K^T$ have large variance — some entries are very large positive, others very large negative. When softmax sees very large values, it saturates: one entry gets nearly all the probability mass and the rest get nearly zero. That kills gradients (the gradient of softmax in the saturated regime is essentially zero). Dividing by √d_k controls the variance and keeps softmax in its useful regime where many tokens get meaningful weight.

MULTI-HEAD ATTENTION

Instead of doing one big attention with d-dimensional Q, K, V, the transformer splits them into h heads, each of dimension d/h, runs attention independently in each head, and concatenates the outputs. Why? Because each head can specialize: one head might learn to track syntactic agreement, another semantic similarity, another positional relationships. Splitting the representation lets the model attend to different things in parallel without one type of attention drowning out the others.

POSITIONAL ENCODING

Self-attention is permutation-invariant — shuffle the tokens and you get the same outputs (just shuffled). That is a problem for language, where order matters. The fix is positional encodings: vectors that encode the position of each token, added to the token embeddings before they enter the first attention layer. The original paper used fixed sinusoidal encodings; later models use learned positional embeddings or rotary positional embeddings (RoPE).

THE FULL BLOCK

A transformer block = multi-head self-attention + add-and-norm + feed-forward network + add-and-norm. The "add" is a residual connection (sum the block's input with its output) which keeps gradients flowing through deep stacks. The "norm" is layer normalization. The feed-forward is two linear layers with a nonlinearity in between, applied independently to each token position. Stack these blocks (12, 24, 96, 175 — the number is the model's signature) and you have a transformer.

WHAT THE PROF EMPHASIZED

The professor emphasized that Q, K, V are not built into the input — they are learned linear projections from the same input embeddings. This is what makes self-attention "self": one input gives rise to all three.

Embeddings get projected up to a higher dimension (d_model, often 768 or 1024) before attention runs. The higher-dimensional space gives the dot products room to encode many distinct relationships without colliding.

QUICK RECALL

Q What problem with RNNs did the transformer solve?	**A** Sequential training (no parallelism) and degradation of long-range dependencies through long chains of nonlinearities.
Q What are Q, K, V intuitively?	**A** Q = what I'm asking for; K = what I contain (label for matching); V = the content that gets passed on when I'm attended to.
Q Write the scaled dot-product attention formula.	**A** $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$
Q Why divide by $\sqrt{d_k}$?	**A** To control the variance of dot products so softmax doesn't saturate — without it, gradients vanish and the model can't learn.
Q What are the dimensions of the attention matrix $Q \cdot K^T$ for n tokens?	**A** n × n — one similarity score for every pair of positions in the sequence.
Q What is multi-head attention and why is it useful?	**A** Splitting Q, K, V into h smaller heads, running attention in each, and concatenating. Each head can specialize on a different type of relationship.
Q Why does the transformer need positional encoding?	**A**

	Because self-attention is permutation-invariant — without position info, the model can't tell whether "dog bites man" or "man bites dog" was intended.
Q What are the two sublayers in a transformer block?	**A** Multi-head self-attention, then a position-wise feed-forward network — each followed by residual connection and layer normalization.

LECTURE

05

Self-Attention vs Cross-Attention

The only thing that changes is where Q, K, V come from

CORE IDEA

Self-attention and cross-attention use the exact same mathematical operation — scaled dot-product attention with softmax($Q \cdot K^T/\sqrt{d_k}$)·V. The only difference is where the three vectors come from. In self-attention all three come from the same sequence; in cross-attention the queries come from one sequence and the keys and values come from another. That single change is what lets one part of a model condition on another — the encoder feeding the decoder, an image feeding a caption, a question feeding a paragraph of text.

SELF-ATTENTION

In self-attention, every token attends to every other token in the same sequence, including itself. Take the sentence "The dog sat on the sofa." When the model is computing the representation for the word "sat", self-attention asks: which other words in this sentence are most relevant to understanding what "sat" means here? It will probably find that "dog" matters most — that is the subject — and that "sofa" matters second. Every position is aware of the entire input.

Q, K, and V are all linear projections of the same input sequence. If your sequence has n tokens, you get an n × n attention matrix where entry (i, j) tells you how much token i attends to token j. This is the building block of BERT, the encoder side of the original transformer, and the only attention used in decoder-only models like GPT (with a causal mask).

CROSS-ATTENTION

Cross-attention links two different sequences. The queries come from sequence A — the one that wants to look something up — and the keys and values come from sequence B — the one being looked at. The classic example is translation. The decoder is generating the French sentence, token by token. At each step, it needs to look back at the English source to decide what to translate next. The decoder's current hidden state becomes Q, and the encoder's output (the encoded English) becomes K and V. The attention matrix is now n_decoder × n_encoder — French queries attending to English keys.

INTUITION

Self-attention: a group of friends in a room talking among themselves — every person can listen to every other person in the same room. Cross-attention: one person in the room is on a phone call

with someone outside — they can ask questions of that outside person and pull information across the wall.

Cross-attention is also what powers vision-language models. In a model that captions images, the text decoder issues queries while the image encoder's features serve as keys and values. The decoder asks "what should the next word be?" and gets back a weighted blend of visual features that are most relevant to answering. In Stable Diffusion, the U-Net's image features are the queries and the text embedding is the keys and values — that is how the text prompt steers the image generation.

SIDE BY SIDE

Property	Self-Attention	Cross-Attention
Source of Q	Sequence X	Sequence A (the asker)
Source of K, V	Sequence X (same)	Sequence B (the source)
Attention matrix size	n × n	n_A × n_B
Used in	BERT, GPT, encoder side of transformer	Decoder-encoder attention, VLMs, Stable Diffusion U-Net
Purpose	Build context-aware representations of one sequence	Let one sequence condition on another

WHAT THE PROF EMPHASIZED

The professor walked through a medical imaging paper that uses cross-attention to fuse two image modalities — the queries came from one scan type and the keys/values from another, letting the model learn how features from one modality should attend to features from another. The point: cross-attention is the general-purpose tool for letting any source of information condition on any other.

EXAM TIP

If asked to distinguish self vs cross attention, the cleanest answer is: "Same mathematical operation. In self-attention, Q, K, V are all projected from the same input sequence. In cross-attention, Q comes from one sequence and K and V come from another."

If asked where cross-attention appears, the safe list: encoder-decoder attention in the original transformer, vision-language models (image as K/V, text as Q), and conditioning blocks in Stable Diffusion (image features as Q, text embedding as K/V).

QUICK RECALL

Q	A

What is the only difference between self-attention and cross-attention?	Where Q, K, V come from. Self: all from the same sequence. Cross: Q from one, K and V from another.
Q In an encoder-decoder transformer doing translation, where do the cross-attention Q and K, V come from?	**A** Q from the decoder (the side generating the target), K and V from the encoder output (the source sentence).
Q What is the shape of the cross-attention matrix?	**A** n_query × n_key — rows indexed by the asking sequence, columns by the source sequence.
Q Name two systems where cross-attention is essential.	**A** Encoder-decoder neural machine translation, and Stable Diffusion (text embedding as K/V conditioning the U-Net's image features as Q). Also any vision-language model.
Q Can self-attention be used in a decoder?	**A** Yes — every token attends to previous tokens (with a causal mask blocking future positions). This is how GPT works.

LECTURE

06

Diffusion Models

Add noise step by step, then learn to remove it

CORE IDEA

A diffusion model defines a fixed forward process that gradually adds Gaussian noise to a data sample over T steps, until at step T the sample is pure noise. It then trains a neural network to reverse this process — to take a noisy sample at step t and predict the slightly less noisy sample at step t−1. Once trained, you start from pure noise and apply the reverse process T times, and out comes a brand new sample that looks like it came from your training distribution. Diffusion has now overtaken GANs as the dominant approach to high-quality image generation.

THE FORWARD PROCESS

The forward process q is fixed — it has no learned parameters. At each step it takes the current sample x_{t-1} and adds a small amount of Gaussian noise to produce x_t. After many steps, the original signal is completely destroyed and you are left with noise from N(0, I). The amount of noise added at step t is governed by a schedule β_t (the variance schedule), which is typically small at the start and grows toward the end.

FORMULA

$$q(x_t \mid x_{t-1}) = N(x_t ; \sqrt{1 - \beta_t} \cdot x_{t-1} , \beta_t \cdot I)$$

There is a beautiful property: because the noise at each step is Gaussian, you can collapse the whole chain into a single closed-form expression. Define $\bar{\alpha}_t$ as the cumulative product of $(1 - \beta_s)$ for s up to t. Then you can jump from x_0 to x_t in one step:

FORMULA

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \varepsilon \quad \text{where} \quad \varepsilon \sim N(0, I)$$

This is what makes diffusion practical to train. Without it you would need to simulate the whole forward chain to get a noisy sample at step t. With it, you just sample a t uniformly, draw an ε, and produce x_t in one operation. The professor walked through this derivation step by step — adding two Gaussians gives another Gaussian, and you keep substituting the recursion until you get the closed form.

THE REVERSE PROCESS

The reverse process is what you actually train. A neural network — usually a U-Net — takes the noisy x_t and the timestep t as input, and predicts the noise ε that was added. Knowing ε lets you back out

x_{t−1} via the formula. The training objective is simple: mean squared error between the true noise and the predicted noise.

FORMULA

$$L_{simple} = E_{t, x_0, \varepsilon} [\| \varepsilon - \varepsilon_\theta(x_t , t) \|^2]$$

INTUITION

Diffusion turns a hard problem into a sequence of easy ones. Generating a sharp image from noise in one step is wildly hard — that is what GANs try to do. Generating a slightly less noisy image from a noisy one is much easier. Diffusion just does the easy version a thousand times and gets the same result — more reliably, with a stable training loss, no minimax game, and no mode collapse.

DDPM SAMPLING ALGORITHM

Sampling from a trained diffusion model:

- Start with x_T sampled from N(0, I) — pure noise.
- For t = T, T−1, …, 1, predict the noise $\varepsilon_\theta(x_t, t)$ with the network, plug it into the reverse-step formula to get x_{t-1}, and add a small amount of fresh noise.
- After T steps, x_0 is your generated sample.

T is a hyperparameter — original DDPM used 1000, which is slow. Later work (DDIM, classifier-free guidance with fewer steps, distilled samplers) brought it down to 20 or even fewer at inference.

STABLE DIFFUSION: LATENT DIFFUSION + TEXT CONDITIONING

Stable Diffusion makes two crucial modifications to make diffusion practical for real-world image generation. First, it runs the diffusion process not in pixel space but in the latent space of a pretrained autoencoder. The autoencoder compresses, say, a 512×512 image down to a 64×64 latent grid. Diffusion runs on the much smaller latent grid, then the decoder upsamples to a full image at the end. This is roughly 64× cheaper than diffusing in pixel space.

Second, it conditions the U-Net on a text prompt. The text is encoded by a frozen language model (CLIP text encoder), and the resulting text embedding is fed into the U-Net via cross-attention layers. At every level of the U-Net, image features query the text embedding, so the denoising process is steered toward an image that matches the prompt. This is exactly the cross-attention pattern from Lecture 5: image as Q, text as K and V.

WHAT THE PROF EMPHASIZED

The professor stressed that the forward process has no learned parameters — it is just adding noise according to a fixed schedule. Only the reverse process (the U-Net) is trained.

The choice of variance schedule β_t matters. Linear schedules work but cosine schedules (used in improved DDPM) preserve more signal in the early steps and produce better samples.

EXAM TIP

If asked to compare diffusion to GANs: stable training (no minimax), better mode coverage, but slower sampling (many forward passes through the network). Quality is typically higher than GANs at the same parameter count.

If asked what the network actually predicts: the noise ε, not the clean image. This parameterization gives the simplest loss and works best in practice (Ho et al., 2020).

If asked why diffusion runs in latent space in Stable Diffusion: pixel-space diffusion is computationally infeasible at high resolution; the latent space of a pretrained autoencoder retains the perceptually meaningful information at a fraction of the cost.

QUICK RECALL

Q Is the forward diffusion process learned or fixed?	**A** Fixed — it just adds Gaussian noise according to a predefined variance schedule β_t. No parameters.
Q What does the neural network actually predict in DDPM?	**A** The noise ε that was added to produce x_t — not the clean image directly.
Q Write the closed-form forward sampling.	**A** $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \varepsilon$, where $\bar{\alpha}_t = \prod(1 - \beta_s)$ up to t and $\varepsilon \sim N(0, I)$.
Q Write the simple training loss.	**A** $L = E[\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2]$
Q What two changes does Stable Diffusion make over plain DDPM?	**A** (1) Diffusion runs in the latent space of a pretrained autoencoder, not pixel space. (2) The U-Net is conditioned on text via cross-attention with a frozen text encoder.
Q Why is diffusion training more stable than GAN training?	**A** It's a single supervised regression objective (predict the noise), not a minimax game. No alternating updates, no equilibrium chasing, no mode collapse.
Q What is the cost of diffusion's stability?	**A** Sampling is slow — you need to run the network many times (T steps), versus a single forward pass for a GAN.

LECTURE

07

Embeddings, LLMs & VLMs

Vector spaces, model families, the modality bridge

CORE IDEA

An embedding is a dense vector representation of something — a word, a sentence, an image, a row of a database — chosen so that semantically similar things end up near each other in the vector space. Embeddings are how language models, vision models, and retrieval systems all communicate. A large language model (LLM) is a transformer trained on vast amounts of text to predict the next token. A vision-language model (VLM) is a model that handles both modalities, typically by mapping images and text into a shared embedding space so the same model can reason about both.

WHAT AN EMBEDDING ACTUALLY IS

Think of every word in your vocabulary getting assigned a point in, say, 768-dimensional space. The training objective (whatever it is — predict the next word, predict masked words, contrastive learning) shapes the geometry of that space. Words that play similar roles end up in similar places: "king" and "queen" close together, "Paris" and "London" close together, "running" and "walking" close together.

The famous arithmetic — king − man + woman ≈ queen — is a property of word2vec-style embeddings. It works because the training objective has carved the space into directions that correspond to semantic dimensions: the "royal" direction, the "gender" direction, the "plural" direction. Modern transformer embeddings are richer (they are contextual — the same word gets a different vector in a different sentence) but the underlying idea is the same: semantically meaningful geometry.

INTUITION

An embedding is a map. Just as a city map turns the rich physical reality of London into a flat picture where nearby points correspond to nearby places, an embedding turns the rich semantic reality of language into a vector space where nearby points correspond to similar meanings. The map loses information, but it preserves what you need.

FROM EMBEDDINGS TO LLMS

A large language model is, at its simplest, a transformer (typically decoder-only) trained on a huge corpus of text with the objective of predicting the next token given the previous tokens. The first thing the model does to any input is look up each token's embedding from a learned embedding table. Those embeddings flow through dozens of transformer blocks. The final hidden state for the last token is projected back through an output embedding (often the same matrix as the input embedding,

transposed) to produce a probability distribution over the entire vocabulary. The token with the highest probability is the predicted next token.

Train this on a trillion tokens of internet text and you get GPT. The same architecture, scaled up, becomes Llama, Claude, Gemini. The differences between these models live in the data, the training procedure (RLHF, constitutional AI, DPO), the safety stack, and engineering — not in the basic architecture.

VISION-LANGUAGE MODELS

A VLM extends an LLM with the ability to understand images. The dominant pattern: train an image encoder (often a vision transformer) to produce image embeddings that live in the same vector space as text embeddings — or train a small adapter that maps image features into the LLM's token space, so the LLM can treat image "tokens" interchangeably with text tokens.

Once images and text share an embedding space, downstream tasks become embedding-similarity problems. Image-text retrieval: given a text query, find the image whose embedding is closest. Visual question answering: feed image tokens and text tokens into a transformer, ask it to generate an answer. Image captioning: same as next-token prediction, but the prefix is the image.

EMBEDDING USE CASES YOU WILL HIT OFTEN

- **Semantic search:** embed your query and your documents into the same space, return the documents whose embeddings have the highest cosine similarity to the query.
- **Retrieval-augmented generation (RAG):** use semantic search to fetch relevant passages, then feed them into an LLM to ground its answer (covered in detail in Lecture 13).
- **Clustering and visualization:** embed your data, run k-means or t-SNE/UMAP to see structure.
- **Recommendation:** embed users and items, recommend items whose embeddings are closest to the user's embedding.
- **Zero-shot classification:** embed both the input and a list of candidate labels, pick the closest label.

WHAT THE PROF EMPHASIZED

The professor noted that token embeddings are projected up to a higher dimension before attention runs — typically 768 or 1024 — because the higher-dimensional space gives the dot products room to encode many distinct semantic relationships without colliding.

Modern embeddings are contextual: BERT and GPT produce a different vector for "bank" in "river bank" versus "savings bank", because the surrounding tokens influence each other through self-attention. This is a major step up from word2vec, which gave each word one fixed vector.

QUICK RECALL

Q	A
What is an embedding?	

	A dense vector representation of an object (word, sentence, image, etc.) where geometric proximity corresponds to semantic similarity.
Q Why is the famous king − man + woman ≈ queen relationship possible?	**A** Because word2vec-style training objectives carve the embedding space into directions that correspond to semantic axes, and gendered terms differ along the same direction as royal terms differ from common terms.
Q What is the difference between a static embedding (word2vec) and a contextual embedding (BERT)?	**A** Static: each word always has the same vector. Contextual: the embedding depends on the surrounding sentence, so polysemous words like "bank" get different vectors in different contexts.
Q What is the basic objective an LLM is trained on?	**A** Next-token prediction — given a sequence of tokens, predict the next token. Cross-entropy loss against the true next token.
Q What does a vision-language model do at the architectural level?	**A** It maps images and text into a shared embedding space (or projects image features into the LLM's token space), so the same transformer can attend across modalities.
Q Name three downstream tasks you can solve once you have good embeddings.	**A** Semantic search, retrieval-augmented generation, zero-shot classification (also clustering, recommendation, anomaly detection).

LECTURE

08

BERT

Bidirectional encoder, MLM, NSP, and the pretrain-finetune recipe

CORE IDEA

BERT — Bidirectional Encoder Representations from Transformers — was the model that proved you could pretrain a single language model on raw text and then fine-tune it on almost any NLP task with state-of-the-art results. Its two key innovations: bidirectional context (every token sees both left and right neighbors, unlike GPT which only sees left), and a clever pretraining objective called Masked Language Modeling that makes bidirectional training possible. BERT is the encoder side of the transformer — no decoder, no generation. Its job is to produce rich contextual representations of text.

WHY BIDIRECTIONAL?

GPT-style models read text left to right and predict the next token. The token at position i sees only positions 1 to i−1. That is fine for generation but limiting for understanding: when humans read "the bank was steep", we use "steep" to disambiguate "bank" as a riverbank, not a financial institution. A left-to-right model has already committed to a representation of "bank" before it sees "steep". Bidirectional means every token's representation depends on both its left and right context — a much richer signal for tasks like classification, question answering, and named-entity recognition.

But you cannot train a bidirectional model with next-token prediction, because every token's prediction would trivially see its own future context. BERT's solution: hide some tokens entirely, and ask the model to fill them in.

MASKED LANGUAGE MODELING (MLM)

BERT randomly selects 15% of tokens in each input sentence and replaces them as follows:

- 80% of the time, replace with a special [MASK] token.
- 10% of the time, replace with a random token from the vocabulary.
- 10% of the time, leave the token unchanged.

The model is then trained to predict the original token at each of the chosen positions. The loss is cross-entropy, and only the chosen positions contribute — unmasked positions are not penalized.

Why this 80/10/10 split? The professor went into the reasoning. The pure-mask approach would create a train-test mismatch: at fine-tuning time there are no [MASK] tokens, but the model would have learned to expect them. So 10% of the time the model has to predict an unchanged token (it doesn't know whether the token in front of it is the original or a corruption), and 10% of the time it has to predict the right token even though a random wrong token has been substituted. This forces the model

to maintain a useful representation of every input position, not just the masked ones, and closes the train-test gap.

INTUITION

MLM is fill-in-the-blank at scale. By forcing the model to predict missing words from their full context, you make it learn the structure of language without needing labels — just raw text. That is the magic of BERT: a single training run on Wikipedia and BookCorpus produced a model that could be quickly adapted to dozens of downstream tasks.

NEXT SENTENCE PREDICTION (NSP)

BERT also has a second pretraining task. The model is fed two sentences A and B separated by a special [SEP] token, with a [CLS] token at the start. Half the time B is the actual sentence following A in the original text; half the time B is a random sentence from somewhere else. The model uses the final hidden state of the [CLS] token to predict which case it is — a binary classification. The intuition is that some downstream tasks (question answering, natural language inference) require understanding relationships between two pieces of text, and NSP gives the model practice at this.

Later research (RoBERTa) showed that NSP is actually not very useful and that removing it improves performance. But the [CLS] token convention from BERT lives on — it is still the standard way to extract a single fixed-size representation of an input sequence for classification.

FINE-TUNING BERT

After pretraining, you fine-tune BERT on whatever task you actually care about. The pattern is:

- Take the pretrained BERT.
- Add a small task-specific head on top (a single linear layer for classification, two linear heads for span prediction in QA, a per-token classifier for NER).
- Train on your labeled task data, updating either just the head or the whole model with a small learning rate.

Because BERT's pretrained weights already encode a lot of linguistic structure, fine-tuning typically converges in just a few epochs on a few thousand labeled examples — orders of magnitude less data than training from scratch would need. This pretrain-finetune recipe is the dominant paradigm in NLP and inspired everything that came after.

BERT'S ARCHITECTURE IN NUMBERS

Variant	Layers	Hidden size	Heads	Parameters
BERT-base	12	768	12	110M
BERT-large	24	1024	16	340M

WHAT THE PROF EMPHASIZED

The professor explained that during MLM training, the cross-entropy loss is only computed at the masked positions — that is where the model is being asked to predict something.

He also noted BERT was a turning point: "It is considered a kind of revolution in the field of NLP when this model came." Almost every encoder model since (RoBERTa, ALBERT, ELECTRA, DistilBERT, DeBERTa) is a refinement of the BERT recipe.

EXAM TIP

Common exam question: "What is the difference between BERT and GPT?" Answer: BERT is encoder-only, bidirectional, trained with MLM, used for understanding tasks (classification, QA, NER). GPT is decoder-only, left-to-right (causal), trained with next-token prediction, used for generation.

If asked about the 80/10/10 split, the key insight is that pure masking creates a train-test mismatch — at fine-tuning the [MASK] token never appears, so the model needs to learn good representations of unmasked tokens too.

QUICK RECALL

Q What does BERT stand for?	**A** Bidirectional Encoder Representations from Transformers.
Q Is BERT a transformer encoder, decoder, or both?	**A** Encoder only — it produces representations, it does not generate text autoregressively.
Q What are BERT's two pretraining objectives?	**A** Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).
Q In MLM, what fraction of tokens are masked, and how exactly?	**A** 15% of tokens are selected. Of those: 80% replaced with [MASK], 10% replaced with a random token, 10% left unchanged.
Q Why is the 80/10/10 mix used instead of always masking?	**A** To prevent a train-test mismatch: at fine-tuning there are no [MASK] tokens, so the model must learn good representations of all positions, not just masked ones.
Q What is the [CLS] token used for?	**A**

	It is the first token of every input. Its final hidden state is used as a fixed-size representation of the whole input for classification tasks.
Q What is the standard recipe for using BERT on a downstream task?	**A** Take pretrained BERT, add a small task-specific head (often a single linear layer on the [CLS] embedding), and fine-tune on labeled task data with a small learning rate.

LECTURE

09

GPT & State Space Models

Decoder-only generation, and the SSM alternative to attention

CORE IDEA — GPT

GPT — Generative Pretrained Transformer — is the decoder-only, left-to-right counterpart to BERT. It is trained to do one thing: given the previous tokens, predict the next one. That sounds modest, but at scale it produces models that can answer questions, write code, summarize documents, and carry on conversations. The architecture is simply a stack of transformer decoder blocks with causal (masked) self-attention, where each token can only attend to itself and earlier tokens — never to future ones.

CAUSAL MASKING

In GPT-style self-attention, the attention matrix has its upper triangle set to negative infinity before the softmax. After softmax, those entries become zero — meaning a token at position i can attend to positions 1 through i, but not to positions i+1 through n. This is what makes the model autoregressive: at training time you feed in the entire sequence at once and predict the next token at every position in parallel, but each position only sees its own past. At inference time you generate tokens one at a time, feeding each new token back into the model to produce the next.

FORMULA

```
mask[i][j]  =  -∞   if   j > i      (block future tokens)
p( x_t | x_<t )  =  softmax( h_t · W_out )
```

WHY GPT TOOK OVER

BERT is great for understanding, but it cannot generate. GPT is great for generation, and it turns out generation is more flexible than people initially realized. Once you have a model that can produce text given a prefix, you can solve almost any task by phrasing it as text completion: classification becomes "This review is: ___", translation becomes "English: hello, French: ___", question answering becomes "Q: ... A: ___". This is the in-context learning insight that powered GPT-3 and everything after.

BERT requires task-specific fine-tuning. GPT can often solve a task just by being prompted with a few examples — no gradient updates at all. As models scaled, this difference became decisive.

THE PROBLEM WITH TRANSFORMERS

The core problem with the transformer is the cost of attention: $O(n^2)$ in sequence length. Doubling the context length quadruples the compute and memory of the attention matrix. For sequences of millions of tokens — entire books, codebases, video frames — this becomes infeasible. Flash Attention (Lecture

14) reduces the memory cost but does not change the asymptotic compute cost. To fundamentally beat $O(n^2)$, you need a different architecture.

STATE SPACE MODELS

State space models (SSMs) are a return to the recurrent idea, but done very carefully. The core equations describe a linear system: a hidden state h_t that updates based on the previous state and the current input, and an output y_t computed from the state.

FORMULA

```
h_t  =  A · h_{t-1}  +  B · x_t
y_t  =  C · h_t   ( + D · x_t )
```

These look just like a vanilla RNN, but the key insight of modern SSMs (S4, then Mamba) is that for the right choice of A, B, C, the recursion can be computed in two ways: as a step-by-step recurrence (linear in n at inference) or as a single global convolution (parallel in n at training). You get the parallelism of a transformer at training time and the linear-time inference of an RNN.

The professor walked through these matrices. A is the state-update matrix, B projects the input into the state, C projects the state into the output. The optional D term is a direct skip connection from input to output and is usually omitted. The forward pass updates h step by step on the math side, but the cleverness is in choosing A so that the recursion can be unrolled into a convolution kernel and applied to the whole sequence in parallel.

INTUITION

Transformers attend to everything in parallel — fast to train, expensive at inference because you have to recompute the attention over a growing context. SSMs maintain a compressed state that gets updated one token at a time — cheap at inference, but tricky to make competitive in quality because the state has to summarize an arbitrarily long history. Mamba bridges both worlds: it makes the SSM state-update input-dependent (selective), so it can choose what to remember and what to forget — recovering much of the expressivity of attention.

SSM VS TRANSFORMER

Property	Transformer	SSM (Mamba)
Compute per token (training)	O(n) per layer (parallel)	O(n) per layer (parallel)
Compute per token (inference)	O(n) — must attend to full context	O(1) — constant per new token
Memory in inference	Grows with context (KV cache)	Constant (fixed state size)
Quality on language	Excellent, the proven leader	Competitive, still catching up
Long-context behavior	Quadratic blowup	Linear, scales gracefully

WHAT THE PROF EMPHASIZED

The professor pointed out that the D term is often dropped from SSM equations — it is just a skip connection and the model usually relies on the state for everything. Don't be confused if you see h_t and y_t equations without a D.

Mamba is the most prominent modern SSM. "Yeah, Mamba is also like a state space model." The key innovation is making A and B input-dependent, so the model selectively remembers or forgets — closer to attention's expressivity while keeping linear scaling.

He emphasized: read the original SSM paper. "It's 36 pages, definitely not possible to cover in one lecture." Reading the first 6–8 pages is enough to make sense of the equations.

QUICK RECALL

Q	A
What is the key architectural difference between BERT and GPT?	BERT is encoder-only with bidirectional self-attention. GPT is decoder-only with causal (masked) self-attention — each token can only attend to previous tokens.
What is GPT's training objective?	Next-token prediction: given tokens x_1 … x_{t−1}, predict x_t. Cross-entropy loss against the true next token.
What does the causal mask do?	Sets the upper triangle of the attention scores to −∞ before softmax, forcing each position to attend only to itself and earlier positions.
What is the asymptotic cost of attention in sequence length?	$O(n^2)$ in both compute and memory — the attention matrix is n × n.
Write the two core SSM equations.	h_t = A·h_{t-1} + B·x_t ; y_t = C·h_t
What is the dual nature of an SSM?	It can be computed as a recurrence (linear-time inference) or as a global convolution (parallel training) — same model, two computational forms.
What does Mamba add over plain SSMs?	Input-dependent (selective) state updates — A and B become functions of the input, letting the model choose what to remember and forget. This recovers much of attention's expressivity.

Q	A
What is the memory cost of SSM inference compared to transformer inference?	SSM: constant — fixed-size state. Transformer: grows linearly with context (KV cache). This is SSM's main advantage for very long contexts.

LECTURE

10

Contrastive Learning (SimCLR)

Pull similar things together, push everything else apart

CORE IDEA

Contrastive learning is a self-supervised method that learns representations by deciding which examples are similar to each other and pulling them together in the embedding space, while pushing dissimilar examples apart. You do not need labels — the "similarity" comes from data augmentation. SimCLR is the canonical paper: take an image, create two random augmented views of it, and train an encoder so the two views map to similar embeddings while every other image in the batch maps to different ones. The result is an encoder that learns visual features as good as supervised pretraining, using only unlabeled images.

THE SIMCLR RECIPE

- Sample a batch of N images from your dataset.
- **Augment each image twice** with random crops, color jitter, blur, and flips. This gives 2N augmented views — N positive pairs (the two views of the same image) and lots of negatives.
- **Pass all 2N views through a shared encoder (typically a ResNet)** to get representations h_i.
- **Project each h_i through a small MLP head g(·)** to get z_i. The contrastive loss is computed in z-space, not h-space — h is what you keep for downstream tasks, z is just for training.
- **Apply the NT-Xent loss:** for each positive pair (z_i, z_j), maximize the cosine similarity between them while pushing them away from all 2N − 2 other views in the batch.

FORMULA

```
L_{i,j}  =  - log [  exp( sim(z_i, z_j) / τ )  /  Σ_{k ≠ i} exp( sim(z_i,
                                                z_k) / τ )  ]
```

The numerator is the similarity between the positive pair. The denominator sums similarities between z_i and every other view in the batch (including the negatives). τ is a temperature hyperparameter that sharpens or smooths the softmax. The loss is small when the positive pair is much closer than any negative — exactly what we want.

INTUITION

Contrastive learning teaches the model: "these two views are the same thing — a cat from two angles, the same dog with different colors. Make their embeddings agree. But this other batch member is a different thing entirely. Make their embeddings disagree." The model is forced to

discover what is invariant about an object across augmentations — and that turns out to be an excellent definition of "semantic content".

THREE CRUCIAL CHOICES IN SIMCLR

The original SimCLR paper made three findings that became standard practice:

- Strong augmentations matter — much stronger than what you'd use for supervised training. Random crops, color distortions, and blur are essential. Without them, the model learns trivial features.
- The projection head g(·) is essential. Compute the contrastive loss on the projected z-space, not the encoder output h-space. The projection head absorbs information that is useful for the contrastive objective but not for downstream tasks, leaving h as a cleaner representation.
- Large batch sizes help, because more negatives per positive give a stronger learning signal. SimCLR used batches of thousands.

WHY IT WORKS

Contrastive learning solves a chicken-and-egg problem in self-supervised learning: how do you learn good features without labels? The answer is to invent your own labels from the data — the augmentation procedure says "these two views are positives, every other image is a negative." Once you have labels, even noisy ones, supervised learning machinery applies. The features that solve the contrastive task — invariance to augmentations, sensitivity to identity — turn out to be the same features that solve downstream tasks like classification.

After pretraining, you discard the projection head, freeze the encoder (or fine-tune it), and use h as input to a downstream classifier. SimCLR showed that on ImageNet, this self-supervised pretraining gave classification accuracy within striking distance of fully supervised training, despite using zero labels during pretraining.

WHAT THE PROF EMPHASIZED

The professor framed contrastive learning as "taking all the data you have, generating samples in such a way that you already know which are positive and which are negative, and learning an embedding where similar examples are close and different examples are far apart." That is the entire idea in one sentence.

EXAM TIP

If asked about SimCLR's three key contributions: (1) strong augmentations, (2) projection head used only during pretraining, (3) very large batch sizes for many negatives.

If asked what NT-Xent is: normalized temperature-scaled cross-entropy. The "normalized" comes from cosine similarity (vectors are L2-normalized), "temperature" is the τ scaling, and "cross-entropy" because the loss has the structure of a softmax over similarities.

QUICK RECALL

Q What problem does contrastive learning solve?	**A** Learning useful representations from unlabeled data — by generating positive pairs from augmentations and treating everything else as negatives.
Q In SimCLR, how are positive pairs created?	**A** By applying two independent random augmentations to the same image. The two augmented views are a positive pair.
Q What is the role of the projection head g(·) in SimCLR?	**A** It maps encoder outputs h to a separate space z where the contrastive loss is computed. After training, the projection head is discarded and only h is used downstream — this gives cleaner features.
Q Write the NT-Xent loss for one positive pair.	**A** $L = -\log[\exp(\mathrm{sim}(z_i, z_j)/\tau) / \Sigma_{k\neq i} \exp(\mathrm{sim}(z_i, z_k)/\tau)]$
Q Why are large batch sizes important in SimCLR?	**A** Because every other example in the batch is a negative — more negatives per positive give a richer learning signal.
Q What augmentations did SimCLR find most important?	**A** Random cropping combined with random color distortion. Either alone is much weaker than both together.

LECTURE

11

Pre-training & Fine-tuning

Transfer learning, SFT, RLHF, and the motivation for PEFT

CORE IDEA

Modern AI is built on a two-stage recipe: pretrain a large model on a huge amount of generic data once, then fine-tune it on a small amount of task-specific data many times. The pretraining stage is expensive but reusable. Fine-tuning is cheap and tailors the same base model to many different tasks. As models grew from millions to billions of parameters, even fine-tuning the whole model became too expensive — which led to parameter-efficient fine-tuning (PEFT), where you update only a tiny fraction of the parameters and freeze the rest.

PRETRAINING

Pretraining is where the model learns general knowledge about the world. For LLMs this means training on hundreds of billions to trillions of tokens of internet text, books, code, and curated corpora. The training objective is simple — usually next-token prediction (for GPT-style models) or masked language modeling (for BERT-style models). The key insight is that learning to predict text accurately requires learning grammar, facts, reasoning patterns, common sense, code structure, and a great deal more, all as side effects of the prediction objective.

Pretraining is enormously expensive. GPT-3-class models take hundreds of GPU-years to train; GPT-4-class models take thousands. The professor was clear: you almost certainly will not pretrain your own LLM. You will start with someone else's pretrained checkpoint.

SUPERVISED FINE-TUNING (SFT)

Once you have a pretrained model, you fine-tune it on labeled examples of the specific task you care about. For a chatbot, this means thousands to tens of thousands of (prompt, ideal response) pairs collected from human writers. The model is trained on these the same way it was pretrained — next-token prediction — except now the loss only penalizes the response tokens, not the prompt tokens.

SFT teaches the base model how to follow instructions, what tone to use, what topics to refuse, and what format to output. A pretrained-only LLM trained on raw internet text will happily continue a question with more questions, or generate a list of plausible-looking answers without committing to one. SFT is what makes it actually answer.

PREFERENCE FINE-TUNING (RLHF / DPO)

After SFT, models are typically fine-tuned further on human preferences. The classic recipe (RLHF — reinforcement learning from human feedback) has two phases. First, train a reward model: present human raters with two model outputs for the same prompt and have them pick which they prefer; train a small model to predict their preferences. Second, use that reward model as a reward signal in reinforcement learning, fine-tuning the LLM with PPO to maximize the predicted reward while staying close to the SFT model (a KL penalty).

RLHF is finicky. Simpler alternatives like DPO (Direct Preference Optimization) skip the explicit reward model and the RL stage, instead converting the preference data directly into a supervised loss. DPO has become the dominant approach in practice.

INTUITION

Pretraining is school. SFT is your first job orientation. Preference fine-tuning is the daily feedback from your manager that turns you from competent into actually useful. Each stage builds on the last, and you can't skip stages.

THE COST PROBLEM THAT MOTIVATED PEFT

Full fine-tuning means updating every weight in the model. For a 7B-parameter LLM, that is 7 billion gradients, 7 billion optimizer states (often 2× or 4× the parameters for Adam), and a forward + backward pass over the full model. Memory becomes the bottleneck — you cannot fit this on a single consumer GPU. And if you want to maintain ten different fine-tuned versions of the same base model for ten different customers, you have to store ten full copies of the 7B weights — about 14 GB each at half precision.

PEFT — Parameter-Efficient Fine-Tuning — solves this by freezing the base model and only training a small number of additional parameters. There are several flavors, but the most successful is LoRA (next lecture), which trains just a few million parameters per task instead of billions. The fine-tuned weights become a small "adapter" that you can load on top of the frozen base.

THE SPECTRUM OF FINE-TUNING

Approach	Updates	Memory	Use case
Full fine-tuning	All parameters	Very high	When you have huge labeled data and big compute
Last-layer fine-tune	Only the head	Low	Linear probing — fast, weak
Adapter layers	Small bottleneck modules inserted in each layer	Low	Original PEFT approach (Houlsby 2019)

LoRA	Low-rank update matrices	Very low	The current default — strong and cheap
QLoRA	LoRA on a 4-bit quantized base	Tiny	Fine-tune 65B models on a single GPU
Prompt tuning	Just a few learned input tokens	Tiny	When you can change inputs but not weights

WHAT THE PROF EMPHASIZED

The professor made the point that the pretrain–fine-tune split is what makes modern AI economically viable: "You can go and say okay no, I will make my own model. That will take a lot of time because you don't have the data, you don't have the resources." Almost no one trains from scratch.

QUICK RECALL

Q What are the three main stages of training a modern chat LLM?	**A** Pretraining (next-token prediction on massive text), supervised fine-tuning (SFT) on instruction-response pairs, and preference fine-tuning (RLHF or DPO) on human preference data.
Q What is the objective of pretraining for a GPT-style LLM?	**A** Next-token prediction with cross-entropy loss, on as much text as possible.
Q What does SFT teach the model that pretraining does not?	**A** How to follow instructions, what tone and format to use, when to answer versus refuse — generally, how to be useful instead of just predictive.
Q How does DPO differ from RLHF?	**A** RLHF trains a reward model and uses RL (typically PPO) to optimize against it. DPO converts the preference data directly into a supervised loss, skipping both the reward model and the RL stage.
Q Why is full fine-tuning of large LLMs expensive?	**A** You need to store gradients and optimizer state for every parameter — typically 4× the model size in memory, plus a full forward and backward pass.
Q What is PEFT and what is its goal?	**A** Parameter-Efficient Fine-Tuning — freezing the base model and training only a small number of additional parameters, dramatically reducing

	memory and storage cost while preserving most of the quality of full fine-tuning.

LECTURE

12

LoRA — Low-Rank Adaptation

Two skinny matrices replace one fat matrix

CORE IDEA

LoRA — Low-Rank Adaptation — is the most successful parameter-efficient fine-tuning method. The insight is that the change required to adapt a pretrained model to a new task has a much lower intrinsic rank than the full weight matrix. Instead of fine-tuning a large d × d weight matrix W, LoRA freezes W and learns a low-rank update ΔW = B·A, where B is d × r and A is r × d for some small rank r (often 8 or 16). The number of trainable parameters drops from d^2 to 2dr — a savings of orders of magnitude. At inference, you can either keep B and A separate (small memory) or merge ΔW into W (no inference cost).

THE MATH

Suppose a layer in the base model computes y = W·x, where W is a d × d matrix. The professor used the example of a 20 × 20 matrix to make this concrete: it has 400 parameters.

LoRA modifies this layer to compute:

FORMULA

```
y  =  W · x   +   ( B · A ) · x          ← the base path plus the LoRA path
                     where  W ∈ ℝ^{d×d}  is  FROZEN
                   A ∈ ℝ^{r×d},  B ∈ ℝ^{d×r}  are  TRAINED
                   r  ≪  d        ( typical r:  4, 8, 16 )
```

Number of trainable parameters: 2 · d · r instead of d^2. For d = 1000 and r = 8, that is 16,000 parameters instead of 1,000,000 — a 60× reduction. At the start of training, A is initialized randomly (small Gaussian) and B is initialized to zero, so B·A = 0 and the model behaves identically to the base model. As training progresses, B fills in and B·A learns the task-specific update.

INTUITION

A 1000 × 1000 matrix has a million entries, but it might have rank only 10 — meaning all the useful structure lives in a 10-dimensional subspace. LoRA's bet is that the difference between a base model and a fine-tuned model is exactly this kind of low-rank object: a small adjustment along a few directions, not a complex perturbation of every weight. The empirical evidence says the bet is mostly right.

WHERE TO APPLY LORA

In a transformer, you do not put LoRA on every weight matrix — that would defeat the savings. The LoRA paper found that applying it to the query and value projections (W_Q and W_V) in each attention layer was sufficient to recover most of the quality of full fine-tuning. Some recipes also apply it to W_K and W_O, or to the feed-forward layers, but Q and V are the consensus minimum.

The result is that for a 7B parameter model, a LoRA adapter might have only 4–16 million trainable parameters — well under 1% of the base. You can train on a single consumer GPU. You can store dozens of fine-tuned variants of the same base model for the cost of one full copy.

INFERENCE MODES

Two ways to run inference with a LoRA-tuned model:

- **Keep B and A separate.** At each layer compute Wx + B(Ax). Slightly slower (one extra small matrix multiply per layer) but you can swap adapters at runtime — load a different LoRA adapter for a different task without touching the base.
- **Merge into the base.** Compute W' = W + BA once and replace W with W'. Now inference is identical in cost to the base model — zero overhead. The price is that you cannot swap adapters anymore.

The merged mode is what makes LoRA strictly better than adapter-layer approaches: those always introduce inference overhead, while LoRA can be merged away.

MEMORY SAVINGS: A CONCRETE NUMBER

From the original LoRA paper, for GPT-3 175B: full fine-tuning requires saving optimizer state for 175 billion parameters, about 1.2 TB. LoRA with rank 4 on Q and V projections trains only 37.7 million parameters — a 10,000× reduction in trainable parameters and roughly 3× reduction in total VRAM during training.

WHAT THE PROF EMPHASIZED

The professor framed LoRA as: "Instead of having this complete matrix, we will have a lower-rank weight matrix." That single substitution is the entire technique.

He noted that LoRA can be initialized so the network starts behaving exactly like the base model — A small random, B zero. This is important because it means LoRA never makes the model worse at the start of training; it can only add capability.

EXAM TIP

If asked to compute the parameter savings: the formula is 2dr versus d^2 (or 2 · d_in · r + 2 · r · d_out for non-square layers). Example: d = 4096, r = 8 → 65,536 trainable LoRA params vs 16,777,216 full params — about 256× fewer.

If asked why B is initialized to zero: so that B·A = 0 at the start, and the LoRA model is identical to the base model on the first step. This guarantees no degradation at initialization.

If asked which projections LoRA is applied to in a transformer: Q and V at minimum (the original recommendation); some recipes also include K and O, or the FFN.

QUICK RECALL

Q What does LoRA stand for?	**A** Low-Rank Adaptation (of Large Language Models).
Q What is the central assumption behind LoRA?	**A** That the weight update needed to adapt a pretrained model to a new task has low intrinsic rank — so it can be expressed as the product of two skinny matrices.
Q Write the LoRA forward pass for a layer.	**A** $y = W \cdot x + (B \cdot A) \cdot x$, where W is frozen and B, A are trained, with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times d}$, and $r \ll d$.
Q How is B initialized and why?	**A** B is initialized to zero, so B·A = 0 at the start of training and the model is initially identical to the base — guaranteeing no degradation from adding LoRA.
Q How many trainable parameters does LoRA introduce per layer?	**A** $2 \cdot d \cdot r$ (versus d^2 for full fine-tuning of that layer). For d = 4096 and r = 8 that's 65,536 instead of 16,777,216.
Q Which projections does the original LoRA paper apply LoRA to?	**A** The query (W_Q) and value (W_V) projections in each attention layer — that was sufficient to match full fine-tuning quality.
Q What are the two inference modes for LoRA?	**A** Keep B and A separate (allows runtime adapter swapping) or merge into the base by computing W' = W + BA (zero inference overhead).

LECTURE

13

QLoRA & RAG

4-bit fine-tuning and retrieval-augmented generation

CORE IDEA — QLORA

QLoRA extends LoRA with aggressive quantization of the frozen base model. Instead of holding the base in 16-bit precision, it is stored in 4-bit precision using a custom data type designed for normally-distributed weights. The LoRA adapters themselves remain in 16-bit. The combination — a 4-bit base plus 16-bit adapters — makes it possible to fine-tune a 65B-parameter model on a single 48GB GPU, something that would otherwise need a cluster. QLoRA is what made fine-tuning Llama-class models accessible to anyone with a serious workstation.

QLORA'S THREE TRICKS

- **4-bit NormalFloat (NF4) quantization.** A custom 4-bit data type whose 16 quantization levels are placed where the values of normally-distributed weights are most likely to fall. Since neural network weights are approximately Gaussian, NF4 wastes very few bits and matches FP16 quality much better than naive 4-bit integer quantization.
- **Double quantization.** The quantization constants used to scale the weights are themselves quantized — saving a small but meaningful amount of additional memory.
- **Paged optimizers.** Optimizer state is held in CPU memory and paged in and out of GPU memory on demand, smoothing out the memory peaks during training and avoiding out-of-memory errors during gradient checkpointing transitions.

During training, the 4-bit weights are dequantized to 16-bit on the fly, the forward and backward pass run in 16-bit, and the gradients flow only into the LoRA adapters (the base weights remain frozen and never get updated). The dequantization is the only extra cost, and it is small.

INTUITION

LoRA reduces what you need to train. QLoRA reduces what you need to store. Together they collapse the memory footprint by an order of magnitude — making single-GPU fine-tuning of 30B–65B models possible. For most practitioners, QLoRA is the default fine-tuning recipe in 2024 and beyond.

The professor noted that without quantization, LoRA still leaves you holding the entire base model in memory in full precision — which dominates the memory cost for very large models. QLoRA is the missing piece that brings the total memory down to something a single GPU can hold.

CORE IDEA — RAG

Retrieval-Augmented Generation (RAG) is a way to give an LLM access to information that is not in its training data — current events, private documents, internal company knowledge. Instead of trying to fine-tune the model on every new piece of information, you keep the LLM frozen and give it relevant context at inference time. When a query comes in, you first retrieve the most relevant documents from a knowledge base, then feed those documents into the LLM along with the query. The LLM generates an answer that is grounded in the retrieved context.

THE RAG PIPELINE

A typical RAG system has two phases — indexing (offline) and querying (online).

Indexing: take all the documents you want the system to know about, split them into chunks of a few hundred tokens each, run each chunk through an embedding model to get a dense vector, and store the (chunk, vector) pairs in a vector database (Pinecone, Qdrant, Weaviate, Postgres + pgvector).

Querying: take the user's question, embed it with the same model, query the vector database for the K chunks whose embeddings are closest to the query embedding (using cosine similarity), assemble those chunks into a prompt along with the original question, and feed the whole thing to the LLM. The LLM generates an answer grounded in the retrieved chunks.

FORMULA

```
answer = LLM( prompt + retrieve_top_k( embed( query ), vector_db ) )
```

THREE RETRIEVAL CATEGORIES

- **Dense retrieval:** embedding-based, captures semantic similarity. "What is the policy on remote work?" matches a document about "work-from-home arrangements" even with no shared keywords.
- **Sparse retrieval (BM25, TF-IDF):** keyword-based, exact-match. Stronger for queries with rare technical terms or names where lexical overlap is the right signal.
- **Hybrid retrieval:** combine dense and sparse scores. Used in production systems because each catches things the other misses.

ADVANCED RAG PATTERNS

Real-world RAG systems use a few extra techniques the professor walked through:

- **Query rewriting:** before retrieval, ask an LLM to rewrite or expand the user's query to make it more retrievable. "thx for help last time" becomes "thank you message regarding previous assistance".
- **Multi-query RAG:** decompose a complex query into several focused sub-queries, retrieve for each, and merge the results. "How did company X perform in 2024 and 2025?" becomes two queries, one per year, with the results combined before generation.

- **Multi-hop RAG:** for queries where the answer to the second question depends on the answer to the first. "Who is the best physics teacher mentioned in last year's reviews?" requires first finding the reviews, then identifying the teachers, then filtering by subject — a chain of retrievals, not a single shot.
- **Reranking:** after the initial retrieval, run a stronger but slower model (a cross-encoder) on the top-K results to reorder them more accurately. The first stage gives you 100 candidates fast; the reranker picks the best 5 carefully.

RAG VS FINE-TUNING — WHEN TO USE WHICH

Question	Use RAG	Use Fine-tuning
The data changes often	✓ Just re-index	✗ Need to retrain
You need to cite sources	✓ Returns chunks	✗ Black box
You need a new style or tone	✗ Doesn't change behavior	✓ Yes
You have a small fixed dataset	Either	Either
You're worried about hallucination on facts	✓ Grounded in retrieved text	Helps but doesn't guarantee
The model needs new skills, not knowledge	✗ Adds context, not capability	✓ Yes

WHAT THE PROF EMPHASIZED

On RAG limitations, the professor acknowledged: "When you cram a very dense search into a very large number of documents with very different kinds of data, vector search struggles more than normal lexical search." The fix is hybrid retrieval and better chunking strategies — RAG is not a silver bullet, and pure semantic search has known failure modes on technical content with specific terminology.

He noted: "You will need some kind of source if you want to chat with a document." RAG is the standard way to make an LLM answer questions over a specific corpus.

EXAM TIP

If asked to draw the RAG pipeline: query → embed → vector search → top-K chunks → LLM with chunks in prompt → answer. Indexing happens once, querying happens per request.

If asked the difference between dense and sparse retrieval: dense uses learned embeddings and captures semantic meaning; sparse uses lexical overlap (BM25) and captures exact keyword matches. Hybrid combines both.

If asked when fine-tuning beats RAG: when you need to teach the model new skills or behaviors (style, format, refusal patterns) rather than new facts. Facts go in via retrieval; skills go in via training.

QUICK RECALL

Q What does QLoRA add over LoRA?	**A** It quantizes the frozen base model to 4-bit precision (using NF4), so you can fine-tune very large models (up to 65B) on a single GPU. The LoRA adapters themselves stay in 16-bit.
Q What are QLoRA's three innovations?	**A** (1) NF4 — a 4-bit data type optimized for normally-distributed weights. (2) Double quantization — quantize the quantization constants. (3) Paged optimizers — page optimizer state between CPU and GPU.
Q What does RAG stand for and what does it solve?	**A** Retrieval-Augmented Generation — gives an LLM access to information beyond its training data by retrieving relevant context at inference time and feeding it into the prompt.
Q What are the two phases of a RAG pipeline?	**A** Indexing (offline): chunk documents, embed, store in vector DB. Querying (online): embed query, retrieve top-K chunks, pass to LLM with the question.
Q Name the three retrieval categories.	**A** Dense (embedding-based, semantic), sparse (BM25, keyword-based), and hybrid (both combined).
Q What is multi-hop RAG?	**A** RAG over a query whose answer requires multiple sequential retrieval steps — the second retrieval depends on the result of the first.
Q When should you use fine-tuning instead of RAG?	**A** When you need to change the model's behavior, style, or skills — not when you need to add new facts. Facts → RAG. Skills → fine-tuning.

LECTURE

14

Flash Attention

IO-aware attention through tiling and online softmax

CORE IDEA

Flash Attention is an exact reformulation of standard attention that runs much faster and uses much less memory — without changing the mathematical result. It does not approximate. The trick is to recognize that the bottleneck of attention on modern GPUs is not floating-point operations but memory bandwidth: shuttling the n × n attention matrix between the GPU's slow high-bandwidth memory (HBM) and its fast on-chip SRAM. Flash Attention uses tiling to compute attention block by block in SRAM, never materializing the full attention matrix in HBM. The result: 2–4× speedup in training, drastic reduction in memory, and the ability to use much longer sequences.

THE GPU MEMORY HIERARCHY

To understand Flash Attention you have to understand the GPU memory hierarchy:

Memory level	Size (A100)	Bandwidth
SRAM (on-chip)	20 MB total	~19 TB/s
HBM (DRAM)	40 GB	~1.5 TB/s

SRAM is roughly 13× faster than HBM but more than a thousand times smaller. The standard implementation of attention reads Q, K, V from HBM, computes the n × n attention scores in HBM, runs softmax in HBM, multiplies by V in HBM, and writes the result back to HBM. The compute on each value is small relative to the bandwidth needed to move it. The whole thing is bandwidth-bound — the GPU's compute units sit idle waiting for HBM.

INTUITION

Imagine a chef who has a small fast counter and a giant slow pantry. The standard recipe makes the chef walk to the pantry for every ingredient, do one operation, walk back, store the result, and repeat. Flash Attention rewrites the recipe: bring a tile of ingredients onto the counter, do everything you can with that tile while it's there, then go fetch the next tile. Same dish, far fewer trips.

TILING AND ONLINE SOFTMAX

Flash Attention's algorithm:

- Split Q, K, V into blocks (tiles) small enough to fit in SRAM.

- For each query block, loop over all key/value blocks. Load the current K, V tile into SRAM. Compute the partial attention scores against the Q tile already in SRAM. Update a running softmax statistic and a running output.
- After visiting all K/V tiles, write the final output for this Q block back to HBM.

The hard part is the softmax. Standard softmax needs to see all the scores at once to compute the normalization constant — the denominator is a sum over the entire row of the attention matrix. Flash Attention solves this with a numerically stable online softmax: as it streams over K/V tiles, it maintains a running max m and a running sum ℓ, and rescales the partial output every time the max changes. By the end of the loop, the running statistics are exactly the same as if you had computed the full softmax in one shot, but you never had to materialize the full row in HBM.

FORMULA

```
After block i,  m_new  =  max( m_old ,  max(S_i) )
ℓ_new  =  e^{m_old - m_new} · ℓ_old   +   Σ e^{S_i - m_new}
O_new  =  ( e^{m_old - m_new} · ℓ_old · O_old   +   e^{S_i - m_new} · V_i
                                  ) / ℓ_new
```

The professor described it as similar to paging in operating systems: you load a tile, do work, evict, load the next. This is exactly the right mental model — Flash Attention treats SRAM as a manually-managed cache and minimizes the round trips to HBM.

WHY IT WORKS

Standard attention has memory complexity $O(n^2)$ because the n × n attention matrix must be fully materialized. Flash Attention has memory complexity $O(n)$ — it never holds more than a few tiles at once. The compute complexity is the same $O(n^2)$ (you cannot avoid the work itself), but because the memory traffic is so much smaller and so much of the work is now done in fast SRAM, the wall-clock time drops dramatically.

The original Flash Attention paper reported 2–4× speedups on training BERT and GPT-2 with no change in numerical result, and 7.6× longer maximum sequence length on the same hardware. Flash Attention 2 (covered briefly in the lecture) further improved this with better work partitioning across thread blocks.

FLASH ATTENTION VS STANDARD ATTENTION

Property	Standard attention	Flash Attention
Mathematical result	Exact $\text{softmax}(QK^T/\sqrt{d})V$	Exact $\text{softmax}(QK^T/\sqrt{d})V$ (identical)
Memory complexity	$O(n^2)$	$O(n)$
HBM accesses	$O(n^2)$	$O(n^2/M)$ where M is SRAM size
Compute complexity	$O(n^2 \cdot d)$	$O(n^2 \cdot d)$ (same)

Implementation	Naive matmul + softmax	Custom CUDA kernel with tiling and online softmax
Speedup in practice	—	2–4× training, more for long sequences

WHAT THE PROF EMPHASIZED

The professor stressed: "Flash Attention is not approximating anything. It is the same softmax($QK^T/\sqrt{d}$)V — just computed differently to avoid materializing the full matrix in HBM." If an exam asks whether Flash Attention changes the model's outputs, the answer is no. It changes the runtime and memory profile, not the math.

He described tiling as "similar to paging" — a useful analogy for students from a computer-science background, and the right way to think about it.

EXAM TIP

If asked why standard attention is slow, the answer is bandwidth, not flops. Modern GPUs are starved for memory bandwidth — they have far more compute than HBM bandwidth — so any algorithm that wastes bandwidth runs slowly even if it does the same arithmetic.

If asked what Flash Attention's three ideas are: (1) tiling — process the attention matrix in blocks small enough to fit in SRAM; (2) online softmax — incremental rescaling so you never need to see the whole row; (3) recomputation in the backward pass — instead of storing the attention matrix for backward, recompute it from the saved Q, K, V tiles, trading flops for memory.

If asked what Flash Attention 2 improves, the answer is better work partitioning across GPU thread blocks and warps — squeezing out more parallelism at the kernel level.

QUICK RECALL

Q	A
Does Flash Attention change the output of attention?	No — it computes the exact same softmax($QK^T/\sqrt{d}$)V. Only the time and memory profile change.
Why is standard attention slow on GPUs?	Because it is bandwidth-bound: the n × n attention matrix has to be moved between slow HBM and the compute units, and those memory transfers dominate runtime.
What is the memory hierarchy that matters for Flash Attention?	SRAM (on-chip, ~20 MB, ~19 TB/s) versus HBM (off-chip, tens of GB, ~1.5 TB/s). Flash Attention keeps work in SRAM.

What are the three core ideas of Flash Attention?	(1) Tiling — process Q, K, V in blocks that fit in SRAM. (2) Online softmax — running max and sum so the normalization can be updated incrementally. (3) Recomputation in backward — recompute the attention matrix instead of storing it.
Q What is the memory complexity of Flash Attention vs standard attention?	**A** Flash: O(n). Standard: $O(n^2)$. The reduction is huge for long sequences.
Q What is the speedup Flash Attention achieves in practice?	**A** 2–4× on standard model training, with much larger gains for very long sequences. Quality is unchanged because the algorithm is exact.

LECTURE

15

Prompt Engineering & LLM Risks

Temperature, top-p, hallucinations, bias, fairness

CORE IDEA

A prompt is the input you give an LLM. Prompt engineering is the practice of crafting prompts that reliably produce the output you want. It also includes choosing the right model and setting the right decoding parameters (temperature, top-p, etc.). Beyond the engineering side, you also have to understand how LLMs go wrong: hallucinations (confident wrong answers), bias (skewed outputs reflecting training data), privacy leaks (memorization of training data), and the limits of fairness in machine-generated text.

CHOOSING A MODEL

Two questions determine your choice of model:

- **Proprietary or open source?** Proprietary models (GPT-4, Claude, Gemini) are usually higher quality and easier to call (one API), but cost money per token, limit you to what the provider offers, and send your data to a third party. Open source models (Llama, Mistral, Qwen, DeepSeek) are free, can run on your own infrastructure, can be fine-tuned, but require GPU resources and engineering effort.
- **Do you need control over the output?** Open source models give you the decoding knobs and the ability to fine-tune; proprietary models hide some of this behind their API.

DECODING PARAMETERS

Generation is sampling. At each step the model produces a probability distribution over the vocabulary. How you turn that distribution into a single chosen token is up to you:

Parameter	What it does	When to set it
Temperature (T)	Divides logits by T before softmax. T → 0 makes output deterministic (greedy); T → 1 keeps the model's natural distribution; T > 1 flattens it.	Low (0.0–0.3) for factual, code, math. High (0.7–1.2) for creative writing, brainstorming.
Top-p (nucleus)	Sample only from the smallest set of tokens whose cumulative probability is at least p.	Typical 0.9–0.95. Cuts off the tail of unlikely tokens without forcing greediness.

Parameter	What it does	When to set it
Top-k	Sample only from the top k most likely tokens.	Less common now; top-p adapts to confidence better.
Frequency / presence penalty	Penalizes tokens that have already appeared, to reduce repetition.	Useful when long generations start to loop.
Max tokens	Hard cap on output length.	Always set, to control cost and runtime.

INTUITION

Temperature is like the model's mood. Cold and deterministic for tax forms; warm and creative for poetry. Top-p is like asking the model to ignore the long tail of weird options — "only consider possibilities you actually think are likely". Most of the time you set temperature alone and leave top-p at 0.9 or 0.95.

PROMPT ENGINEERING TECHNIQUES

- **Zero-shot:** Just ask the question. Works well for simple, common tasks.
- **Few-shot (in-context learning):** Provide a handful of examples in the prompt before the actual question. The model picks up the pattern. Powerful for tasks with consistent format.
- **Chain-of-thought (CoT):** Tell the model to "think step by step" before answering. For multi-step reasoning problems, this dramatically improves accuracy by letting the model use intermediate tokens as scratch space.
- **Role prompting:** "You are an experienced biology professor explaining to a high school student." Sets tone, vocabulary, and level.
- **Output format constraints:** "Respond in JSON with fields name, age, occupation." Improves downstream parseability.
- **Self-consistency:** Generate multiple answers with high temperature and pick the most common one. Trades compute for accuracy on reasoning tasks.

HALLUCINATIONS

A hallucination is when a model generates a confident, fluent answer that is factually wrong. The model is not lying — it is doing exactly what it was trained to do, which is to produce plausible-sounding text. The training objective rewards fluency and consistency with patterns in the training data; it does not reward truth, because the model has no notion of an external world to check against.

Common sources of hallucination:

- The question asks about something not in the training data — the model fills the gap with a plausible guess.

- The question asks about something rare or contested in the training data — the model averages multiple sources and produces a hybrid that matches none of them.
- The model is confidently extrapolating from patterns: famous person + question about their work → invented book title that sounds right but does not exist.
- Long generations drift: each token is conditioned on the previous tokens, including any earlier mistakes, so errors compound.

Mitigations:

- RAG — ground the answer in retrieved documents rather than letting the model rely on its parametric memory.
- Lower temperature — fewer creative leaps.
- Asking the model to cite sources or refuse if it is uncertain.
- Verification — running the answer through a checker model or against a knowledge base.

BIAS AND FAIRNESS

LLMs learn from human-generated text, and human-generated text contains every bias humans have — gender, race, region, profession, religion. The model learns these biases as part of the statistical patterns of the data and reproduces them in its outputs. A model asked to complete "the doctor said" may default to "he"; asked about a stereotypically associated profession, it may produce text that reinforces the stereotype.

Mitigations are imperfect:

- Curating training data to reduce the most blatant problems.
- Fine-tuning on counter-stereotypical examples.
- RLHF with human raters who flag biased outputs.
- Adding guardrails — a separate classifier that screens outputs for bias and refuses or rewrites them.

None of these eliminates bias; they reduce its visibility in surface outputs while leaving the underlying representations largely unchanged. This is an open area of research and a real concern in deployment.

PRIVACY AND MEMORIZATION

Large language models memorize substantial portions of their training data. If the training set contains personal information — phone numbers, social security numbers, private emails scraped from the web — the model can sometimes be coaxed into reproducing it. This is a serious problem for any LLM trained on user data without careful filtering. Mitigations include differential privacy during training, deduplication of the training corpus (memorization is much worse for documents that appear many times), and post-hoc filters that detect and block PII in outputs.

WHAT THE PROF EMPHASIZED

The professor stressed that LLM applications should expose the right knobs to users when meaningful: a brainstorming tool should default to higher temperature, a code generator to lower. ChatGPT-style consumer products hide these knobs and try to infer the right setting from the prompt — that works most of the time but fails when you actually want control.

On hallucinations, his framing: "the model does not actually understand the context. It will use misinterpreted completely." The model is a pattern-matching engine, not a reasoning agent — when patterns run out, it confabulates rather than admitting ignorance.

EXAM TIP

If asked the difference between temperature and top-p: temperature reshapes the distribution (sharper or flatter); top-p truncates the distribution (cut off the unlikely tail). They are usually used together.

If asked what chain-of-thought does, the answer is that it gives the model intermediate "scratch space" — by generating reasoning tokens before the final answer, the model can perform multi-step computation that single-token generation cannot.

If asked why LLMs hallucinate, the cleanest answer is: they are trained to produce plausible text, not true text. The objective rewards fluency and pattern-consistency; truth is not in the loss function.

QUICK RECALL

Q What does temperature do in LLM sampling?	**A** Divides the logits by T before softmax. Lower T sharpens the distribution (more deterministic), higher T flattens it (more random and creative). T = 0 is greedy decoding.
Q What does top-p (nucleus) sampling do?	**A** Restricts sampling to the smallest set of tokens whose cumulative probability is at least p. It cuts off the unlikely tail without forcing greediness.
Q What is few-shot prompting?	**A** Including several example input-output pairs in the prompt before the real question, so the model picks up the pattern from context — no fine-tuning required.
Q What is chain-of-thought prompting?	**A** Instructing the model to produce intermediate reasoning steps before the final answer ("think

	step by step"). It improves accuracy on multi-step problems by giving the model scratch space.
Q What is a hallucination?	**A** A confident, fluent model output that is factually wrong. The model is doing what it's trained to do — produce plausible text — but plausibility is not truth.
Q Name three mitigations for hallucination.	**A** (1) RAG — ground answers in retrieved documents. (2) Lower temperature. (3) Source citation requirements or refusal options. (Also verification with a separate model.)
Q Why are LLMs biased?	**A** They learn from human-generated text, which reflects human biases. The training objective preserves whatever statistical patterns exist in the data, including the unwanted ones.

MASTER CHEAT SHEET

Everything in two pages. Use this for last-minute revision the night before the exam.

GENERATIVE MODEL FAMILIES

Family	Idea	Trained how	Strength	Weakness
GAN	G vs D minimax game	Adversarial; alternating updates	Sharp samples	Mode collapse, unstable
VAE	Encode to a distribution, sample, decode	ELBO = recon − KL	Stable, samplable latent space	Blurry samples
Diffusion	Noise then learn to denoise	MSE on predicted noise	Stable, high quality, mode coverage	Slow sampling
Autoregressive (GPT)	Predict next token	Cross-entropy on next token	General purpose, in-context learning	$O(n^2)$ attention
Encoder LLM (BERT)	Bidirectional MLM	MLM + NSP	Strong for understanding tasks	Cannot generate

CORE FORMULAS

FORMULA

```
GAN:        min_G max_D  E_x[log D(x)] + E_z[log(1 - D(G(z)))]
VAE:        L = E_q[log p(x|z)]  -  KL(q(z|x) ‖ p(z))
Reparam:    z = μ + σ · ε,    ε ~ N(0, I)
Attention:  softmax(Q · Kᵀ / √d_k) · V
Diffusion:  x_t = √(ᾱ_t)·x_0 + √(1-ᾱ_t)·ε
DDPM loss:  L = E[ ‖ε - ε_θ(x_t, t)‖² ]
LoRA:       y = Wx + (BA)x,   B init 0,   r ≪ d
SSM:        h_t = A·h_{t-1} + B·x_t,   y_t = C·h_t
RAG:        answer = LLM( query + retrieve_top_k( embed(query) ) )
```

PEFT FAMILY — AT A GLANCE

Method	What it adds	Trainable params	Best for
Adapter	Bottleneck modules in each layer	~1% of base	Original PEFT, replaced by LoRA
LoRA	Low-rank update BA on Q and V	<1% of base	The current default
QLoRA	LoRA on a 4-bit base (NF4)	<1% + 4-bit storage	Single-GPU fine-tune of huge models
Prompt tuning	Learned prompt tokens	Tiny	When you can't touch model weights

ATTENTION SCALING TECHNIQUES

Technique	What it changes	Effect	Approximate?
Standard attention	—	$O(n^2)$ memory, $O(n^2)$ compute, bandwidth-bound	—
Flash Attention	Tiling + online softmax	$O(n)$ memory, 2–4× speedup, same compute	No (exact)
Flash Attention 2	Better thread-block partitioning	Further speedup	No (exact)
State Space Models	Recurrent state instead of attention	$O(1)$ inference per token, linear training	Different architecture

RAG VS FINE-TUNING — WHEN TO CHOOSE

Need	Choose
Add new factual knowledge	RAG
Add new behaviors / format / style	Fine-tune
Knowledge changes often	RAG
Need source citations	RAG
Reduce hallucinations on facts	RAG (or both)

Match a specific tone	Fine-tune
Custom output schema	Fine-tune (or strong prompting)

DECODING PARAMETER CHEAT SHEET

Goal	Temperature	Top-p	Notes
Code, math, factual QA	0.0 – 0.3	0.9	Near-greedy
Summarization, structured output	0.3 – 0.5	0.9	Slight variation, mostly deterministic
Conversation, default chat	0.6 – 0.8	0.9 – 0.95	ChatGPT default territory
Brainstorming, creative writing	0.8 – 1.2	0.95	Embrace surprise
Multiple diverse samples	1.0+	0.95	Generate many, pick best

FINAL FLASHCARD SPRINT

60 high-yield cards across the whole syllabus. Cover the right column, work through the left, then check.

Q Generative vs discriminative — one line.	**A** Discriminative learns $p(y\|x)$; generative learns $p(x)$ and lets you sample new x.
Q GAN objective.	**A** $\min_G \max_D E[\log D(x)] + E[\log(1 - D(G(z)))]$
Q What is mode collapse?	**A** Generator produces few outputs that fool D, ignoring the rest of the data distribution.
Q VAE loss.	**A** Reconstruction term + $KL(q(z\|x) \| p(z))$
Q Reparameterization trick.	**A** $z = \mu + \sigma \cdot \varepsilon$ with $\varepsilon \sim N(0, I)$, so gradients can flow through μ and σ.
Q Why are VAE samples blurry?	**A** Per-pixel reconstruction loss averages over many plausible reconstructions.
Q Q, K, V — one line each.	**A** Q = what I'm asking, K = what I label myself as, V = what I carry.
Q Scaled dot-product attention formula.	**A** $\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$
Q Why divide by $\sqrt{d_k}$?	**A** To prevent softmax saturation, which would kill gradients.
Q Self vs cross attention — one line.	**A** Self: Q, K, V from same sequence. Cross: Q from one, K and V from another.
Q What does multi-head attention buy?	**A**

	Each head can specialize on a different relationship in parallel.
Q Why does the transformer need positional encoding?	**A** Self-attention is permutation invariant; without position info, word order is lost.
Q What does the diffusion forward process do?	**A** Adds Gaussian noise according to a fixed schedule β_t. Not learned.
Q What does the diffusion network predict?	**A** The noise ε that was added — not the clean image.
Q DDPM training loss.	**A** MSE between true noise and predicted noise: $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$
Q Two changes Stable Diffusion makes over plain DDPM.	**A** Diffuses in latent space (not pixels), and conditions on text via cross-attention.
Q What is an embedding?	**A** A dense vector where geometric proximity = semantic similarity.
Q BERT vs GPT.	**A** BERT: encoder, bidirectional, MLM, for understanding. GPT: decoder, causal, next-token, for generation.
Q BERT's two pretraining objectives.	**A** Masked language modeling (MLM) and next sentence prediction (NSP).
Q MLM 80/10/10 split.	**A** Of 15% selected: 80% [MASK], 10% random token, 10% unchanged.
Q Why the 80/10/10 mix?	**A** To prevent train-test mismatch where [MASK] never appears at fine-tuning.
Q What does the [CLS] token do in BERT?	**A** Its final hidden state is the input-level representation used for classification.
Q	**A**

What does the causal mask in GPT do?	Sets attention scores to $-\infty$ for future positions, so each token attends only to its past.
Q Asymptotic cost of attention.	**A** $O(n^2)$ in compute and memory.
Q State space model equations.	**A** $h_t = A \cdot h_{t-1} + B \cdot x_t$; $y_t = C \cdot h_t$
Q Mamba's key innovation over plain SSMs.	**A** Input-dependent (selective) state updates — A and B become functions of x.
Q Inference cost: SSM vs Transformer.	**A** SSM: O(1) per token (constant state). Transformer: O(n) per token (KV cache grows).
Q What does SimCLR do?	**A** Self-supervised contrastive learning: two augmented views of an image are positives, all others are negatives.
Q NT-Xent loss in one line.	**A** $-\log[\exp(\mathrm{sim}(z_i, z_j)/\tau) / \Sigma_{k\neq i} \exp(\mathrm{sim}(z_i, z_k)/\tau)]$
Q Three crucial choices in SimCLR.	**A** Strong augmentations, projection head used only at training, large batches.
Q Three stages of training a chat LLM.	**A** Pretraining → SFT → preference fine-tuning (RLHF or DPO).
Q What does SFT teach beyond pretraining?	**A** Instruction-following, tone, format, refusal — how to be useful, not just predictive.
Q LoRA core idea.	**A** Replace ΔW with $B \cdot A$ where B is $d \times r$, A is $r \times d$, $r \ll d$. Freeze W.
Q LoRA forward pass.	**A** $y = W \cdot x + (B \cdot A) \cdot x$
Q Why is B initialized to zero in LoRA?	**A** So $B \cdot A = 0$ at start; the LoRA model is initially identical to the base.

Q LoRA trainable params per layer.	**A** $2 \cdot d \cdot r$ instead of d^2 — orders of magnitude fewer.
Q Which projections does LoRA target?	**A** Q and V (at minimum). Sometimes also K, O, or FFN.
Q Two LoRA inference modes.	**A** Keep B and A separate (swappable adapters) or merge into W (zero overhead).
Q What does QLoRA add over LoRA?	**A** Stores the frozen base in 4-bit (NF4), enabling single-GPU fine-tune of 65B models.
Q QLoRA's three innovations.	**A** NF4 quantization, double quantization, paged optimizers.
Q What does RAG stand for?	**A** Retrieval-Augmented Generation.
Q Two phases of a RAG pipeline.	**A** Indexing (offline: chunk, embed, store) and querying (online: embed query, retrieve, prompt LLM).
Q Three retrieval categories.	**A** Dense (semantic embeddings), sparse (BM25), hybrid (both).
Q What is multi-query RAG?	**A** Decompose a query into several focused subqueries, retrieve for each, merge results.
Q What is multi-hop RAG?	**A** Sequential retrieval where the second query depends on the first answer.
Q RAG vs fine-tuning — facts vs skills.	**A** Facts → RAG. Skills, behavior, style → fine-tuning.
Q Does Flash Attention change the math?	**A** No. Same exact $\text{softmax}(QK^T/\sqrt{d})V$. Only the time and memory profile differ.
Q Why is standard attention slow?	**A**

	Bandwidth-bound: shuttling the n × n matrix between HBM and SRAM dominates.
Q Flash Attention's three ideas.	**A** Tiling, online softmax, recomputation in backward pass.
Q Memory complexity: standard vs Flash.	**A** Standard $O(n^2)$; Flash $O(n)$.
Q Practical Flash Attention speedup.	**A** 2–4× on training, more on long sequences.
Q Temperature in sampling.	**A** Divides logits by T before softmax. Lower = more deterministic, higher = more random.
Q Top-p sampling.	**A** Sample only from the smallest token set whose cumulative probability is ≥ p.
Q Few-shot prompting.	**A** Including example input-output pairs in the prompt before the real question.
Q Chain-of-thought prompting.	**A** Asking the model to think step by step before answering — gives it scratch space for multi-step reasoning.
Q What is a hallucination?	**A** A confident, fluent answer that is factually wrong. The model produces plausible text, not true text.
Q Three mitigations for hallucination.	**A** RAG, lower temperature, source citation / verification.
Q Why are LLMs biased?	**A** They learn from human text, which reflects human biases. The training objective preserves all the patterns, including unwanted ones.
Q Why does LLM memorization happen?	**A** The model has the capacity to fit individual training examples, especially those that appear repeatedly. Mitigations: dedup, differential privacy, output filters.

Q	A
What is the single most important habit for the exam?	When you see a model name, immediately recall: which family it belongs to, what its training objective is, what its loss looks like, and what its key innovation was. Everything else flows from those four.

G O O D L U C K